

Reviewer Pack — Minimal Order of Automorphism Groups and Communication Compl...

Entry a10a2f66a4d8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 09:04:42 UTC

Minimal Order of Automorphism Groups and Communication Complexity with Tropical Motive Homology

Entry ID: a10a2f66a4d8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 09:04:42 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (specifically, tropical motive homology) **Field B** (complexity object): Communication Complexity

Statement:

For any given boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the minimal order of automorphism groups for its associated tropical motive homology group $H(f)$ is linearly correlated with its deterministic communication complexity $C(f)$, such that $\log |\text{Aut}(H(f))| = \Theta(C(f))$.

Rationale (proposer's reasoning):

Tropical motive homology provides a rich algebraic structure over the space of boolean functions, which may capture nontrivial symmetries and dependencies. If the order of automorphism groups is related to communication complexity, it could indicate that certain problems require more information exchange than others due to inherent structural properties.

Taxonomy category: TROPICAL_MOTIVE_HOMOLOGY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): afd60bc52b5daba7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$, at least 80% of randomly generated boolean functions f have a correlation coefficient between $\log |\text{Aut}(H(f))|$ and $C(f)$ greater than or equal to 0.7.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tropical_motive_homology(f):
        # Placeholder implementation of tropical motive homology
        # This is a dummy function and should be replaced with actual computation
        return f

    def automorphism_group_order(homology):
        # Placeholder implementation of automorphism group order
        # This is a dummy function and should be replaced with actual computation
        return len(homology)

    def communication_complexity(f):
        # Placeholder implementation of communication complexity
        # This is a dummy function and should be replaced with actual computation
        return len(f)

    results = []
    for n in range(5, 41):
        instances_tested = 0
        total_log_aut = 0
        total_C = 0
        for _ in range(30):
            f = generate_boolean_function(n)
            homology = tropical_motive_homology(f)
            log_aut = math.log(automorphism_group_order(homology))
            C = communication_complexity(f)
            results.append({"n": n, "log_aut": log_aut, "C": C})
            instances_tested += 1
        total_log_aut += sum(result["log_aut"] for result in results if result["n"] == n)
```

```

        total_C += sum(result["C"] for result in results if result["n"] == n)

mean_log_aut = total_log_aut / len(results)
mean_C = total_C / len(results)
correlation_coefficient = (sum((result["log_aut"] - mean_log_aut) * (result["C"] - mean_C) for
                               result in results) /
                           math.sqrt(sum((result["log_aut"] - mean_log_aut)**2 for result in results) *
                                       sum((result["C"] - mean_C)**2 for result in results)))

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": 40,
    "conjecture_holds": correlation_coefficient >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    if not sys.argv[1:]:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79]
    else:
        seeds = [int(s) for s in sys.argv[1:]]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) /
                                  len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{first_failing_seed}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be verified. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	28196
2	preregistration	ollama_remote	glm4:latest	0	0	27246
3	novelty	ollama_remote	glm4:latest	0	0	16895
4	novelty	ollama_remote	glm4:latest	0	0	15888
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19265
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16571
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	52771
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16003
9	judge	ollama_remote	glm4:latest	0	0	15416

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 208251 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/a10a2f66a4d8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/a10a2f66a4d8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/a10a2f66a4d8.tar.gz (if generated)